



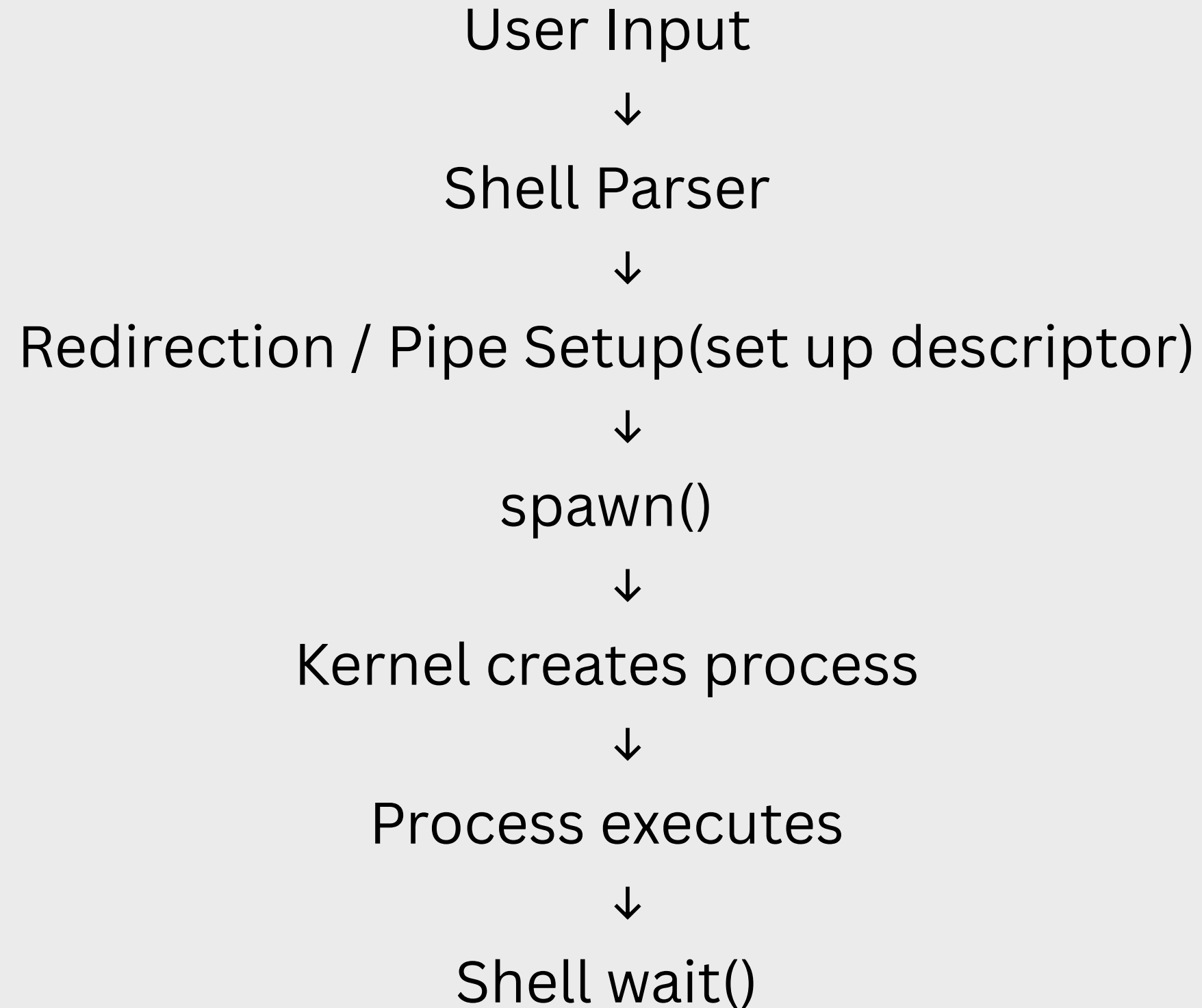
UNIX SHELL

Group 09

OBJECTIVE

- Implement a UNIX-style interactive shell
- Support file and directory operations
- Implement file descriptor abstraction
- Enable I/O redirection (<, >, >>)
- Implement pipelines (|)
- Support recursive directory operations
- Maintain process isolation

SYSTEM ARCHITECTURE OVERVIEW



SYSTEM ARCHITECTURE OVERVIEW

User Space:

- **Shell process** – reads commands, manages environment, handles I/O redirection.
- **Command implementations** (cp, mv, rm, etc.) – perform file operations using system calls.
- **Command parser** – splits input into command + arguments, handles options, pipes, and redirection.

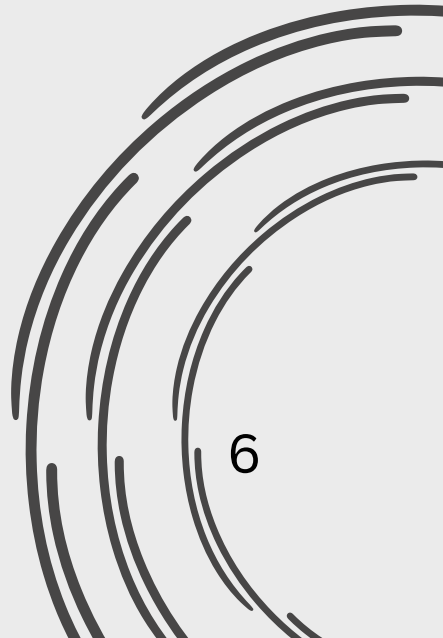
SYSTEM ARCHITECTURE OVERVIEW

Kernel Space

- **File system** (inodes + directories) – manages files, directories, and metadata.
- **File descriptor table** – tracks open files, positions, and access modes.
- **Pipe subsystem** – enables communication between processes.
- **System call handler** – validates user requests and executes syscalls (open, read, write, etc.).
-



IMPLEMENTATION



touch

user/shell/shell.c

```
int shell_touch(int argc, char** argv)
{
    if (argc == 1) {
        printf ("touch failed. No Path. \n");
        return 0;
    }
    int i;
    for (i = 1; i < argc; i++)
    {
        int fd;
        fd = open(argv[i], O_RDONLY);
        if(fd >= 0){
            printf("%s file exist\n", argv[i]);
            close(fd);
            continue;
        } else {
            close(open(argv[i], O_CREATE));
        }
    }
    return 0;
}
```

cd

kern/fs/sysfile.c

```
void sys_chdir(tf_t *tf) //cd
{
    char path[128];
    struct inode *ip;
    int pid = get_curid();
    unsigned int path_len = syscall_get_arg3(tf);
    path_len = path_len < 128? path_len: 127;
    pt_copyin(get_curid(), syscall_get_arg2(tf), path, path_len);
    path[path_len] = '\0';

    if((ip = namei(path)) == 0){
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }
    inode_lock(ip);
    if(ip->type != T_DIR){
        inode_unlockput(ip);
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }
    inode_unlock(ip);
    inode_put(tcb_get_cwd(pid));
    tcb_set_cwd(pid, ip);
    syscall_set_errno(tf, E_SUCC);
}
```


mv

user/shell/shell.c

```
int shell_mv(int argc, char** argv)
{
    if(argc != 3){
        printf("mv: argument invalid.\n");
        return 0;
    }
    char* src = argv[1];
    char* dest = argv[2];
    if(!is_file_exist(src)){
        printf("mv: source file %s does not exist.\n", src);
        return 0;
    }
    if (is_dir(src)) {
        if (is_file_exist(dest) && !is_dir(dest)) {
            printf("mv: cannot move a dir to a file\n");
            return 0;
        }
        _shell_cp(dest, src, 1);
        _shell_rm(src, 1);
    } else {
        _shell_cp(dest, src, 1);
        _shell_rm(src, 0);
    }
    return 0;
}
```

ls

kern/trap/TSyscall/TSyscall.c

```
void sys_ls(tf_t *tf)
{
    uint32_t off, inum;
    struct dirent de;
    uint32_t de_size;
    struct inode * p_inode;
    int user_buf_addr = syscall_get_arg2(tf);
    int buf_len = syscall_get_arg3(tf);
    int cur_pid = get_curid();
    int len;
    char * buf_p = sys_buf[cur_pid];
    struct inode* dp = (struct inode*)tcb_get_cwd(cur_pid);
    de_size = sizeof(de);
    for(off = 0; off < dp->size; off += de_size){
        if(inode_read(dp, (char *)&de, off, de_size) != de_size){
            // size is not legal
            KERN_PANIC("wrong in dir_lookup");
        }
        if(de.inum == 0){
            // free entry
            continue;
        }
        // TODO index may out of bound
        strncpy(buf_p, de.name, strlen(de.name, PAGESIZE));
        buf_p += strlen(de.name, PAGESIZE);
        *(buf_p++) = ' ';
        //dprintf("%s ", de.name);
    }
    *(buf_p - 1) = '\0';
    //dprintf("\n");
    len = buf_p - sys_buf[cur_pid] < buf_len? buf_p - sys_buf[cur_pid]: buf_len;
    pt_copyout(sys_buf[cur_pid], cur_pid, user_buf_addr, len);
    syscall_set_errno(tf, E_SUCC);
    syscall_set_retval1(tf, len);
}
```



THANK YOU